# Kit móvil de atención primaria en salud

## Documento de arquitectura

**Proyecto final - Plataformas I - 2026-2** · Organización `kitsalud-movil-plats1`

Este documento describe el kit tal como está construido, es decir, sus equipos, su topología física y lógica, la segmentación, el direccionamiento IPv4 e IPv6, los nombres DNS, las dependencias entre servicios, el dimensionamiento y las decisiones de diseño con sus razones.

## 1. Contexto y alcance

Una brigada de salud debe desplegar en sitio una infraestructura pequeña que preste servicios clínicos, administrativos y comunitarios **sin depender de Internet**. El kit se compone de **un mini PC** (Beelink EQi12), **un MikroTik CCR2004** que hace de switch y **un punto de acceso Wi-Fi** (TP-Link TL-WA801ND). El mini PC es a la vez el router/firewall del kit y el hipervisor de dos máquinas virtuales, una para los servicios clínicos y otra para los comunitarios. La alimentación segura (UPS) se incluye de forma lógica, porque no se cuenta con el equipo físico. El enlace a Internet del sitio es opcional.

**Contenido para la comunidad.** Los contenidos le sirven tanto a la comunidad como al personal de salud. Por eso, además de la literatura de salud, el kit ofrece una **biblioteca educativa para niños** y contenido de salud y comunitario en **audio y video** (sección 10.5).

El criterio que guía el diseño es el del enunciado, que pide construir una plataforma **pequeña, segura, reproducible y útil** con la menor cantidad de tecnologías posible. Por eso hay un solo sistema operativo (Ubuntu Server 24.04) en el host y en las VMs, dos VLAN físicas, dos VMs y, para cada requisito, la herramienta más simple que lo cumple.

### 1.1 Usuarios, necesidades y aplicaciones

Cada necesidad tiene una aplicación, un nombre fácil de recordar y una red desde donde se usa.

| Usuario | Necesidad | Aplicación | Nombre | Corre en | Red |
|---|---|---|---|---|---|
| **Personal de salud** | Registrar y seguir pacientes | DHIS2 | `pacientes.salud.movil` | clinica01 | Interna |

| Usuario | Necesidad | Aplicación | Nombre | Corre en | Red |
|---|---|---|---|---|---|
| **Personal de salud** | Ver los prerregistros y marcar a quién se atendió | Consulta de prerregistros | `consultas.salud.movil` | clinica01 | Interna |
| **Personal de salud** | Compartir documentos | Recursos SMB de Samba | `archivos.salud.movil` | clinica01 | Interna |
| **Personal técnico** | Ver el estado, las métricas y los logs | Grafana, Prometheus y rsyslog | `monitoreo.salud.movil` | kit01 | Direcciones de administración o NetBird |
| **Personal técnico** | Configurar, respaldar y recuperar | SSH, Ansible y restic | `kit01` | kit01 | Direcciones de administración o NetBird |
| **Comunidad y visitantes** | Conectarse al Wi-Fi | Portal cautivo | `portal.salud.movil` | kit01 | Comunidad |
| **Comunidad y visitantes** | Consultar literatura y contenido educativo | Kiwix | `biblioteca.salud.movil` | comunidad01 | Comunidad |
| **Comunidad y visitantes** | Ver videos y escuchar audio | Jellyfin | `videos.salud.movil` | comunidad01 | Comunidad |
| **Pacientes en sala de espera** | Llenar el prerregistro desde su celular | Formularios | `registro.salud.movil` | comunidad01, y guarda en clinica01 | Comunidad |

# 2. Requisitos y pruebas

## 2.1 Requisitos técnicos

| Requisito | Solución | Dónde |
|---|---|---|
| **DHCPv4 y provisión IPv6** | Kea DHCPv4 con pools y reservas por MAC. radvd anuncia SLAAC con M=0, O=1 y Kea DHCPv6 stateless entrega DNS, dominio y NTP. Servidores con dirección estática | kit01 |
| **DNS interno dual-stack** | BIND9 autoritativo para `salud.movil` (A, AAAA, PTR v4/v6), recursivo solo para las redes internas | kit01 |

| Requisito | Solución | Dónde |
|---|---|---|
| **NTP** | Chrony como servidor (`ntp.salud.movil`). Con Internet se sincroniza con fuentes externas (`pool.ntp.org`); sin Internet sigue como referencia con `local stratum 10`. Escucha solo en la IP de servicios (`.10` y `::10`). Las VMs y el switch sincronizan contra él; el AP no tiene ajustes de hora en su firmware | kit01 |
| **Firewall y segmentación** | Dos VLAN 802.1Q + red de servidores virtual. **nftables** en una tabla `inet` (IPv4 e IPv6), denegar por defecto y registrar. NAT en una tabla aparte | kit01, sw01 |
| **Portal cautivo** | Portal propio con nginx y un set de MAC en nftables con expiración. Cubre IPv4 **e IPv6** | kit01 |
| **Aplicación de pacientes** | DHIS2 (Docker) + PostgreSQL, con cuentas locales y el dashboard "Brigada de salud" (atenciones, fiebre, motivos, sexo y edad), que se actualiza cada 30 minutos | clinica01 |
| **Portal de literatura** | Kiwix (salud y contenido infantil) y Jellyfin (audio y video de salud y para la comunidad) | comunidad01 |
| **Formularios de prerregistro** | App propia mínima y asistida. Los datos se guardan en PostgreSQL de clinica01 y los consulta solo el grupo AD `clinicos`, en `consultas.salud.movil` con login LDAPS | comunidad01 → clinica01 |
| **Almacenamiento compartido** | SMB3 con autenticación AD (Kerberos o NTLMv2), con los recursos `archivos` (documentos y exportaciones, solo del grupo `clinicos` y con cifrado obligatorio) y `contenido` (medios y ZIM que se publican en comunidad01, con escritura de `clinicos` y `admins`). Los grupos y usuarios tienen gid y uid fijos (RFC2307), así un backup restaurado conserva sus dueños | clinica01 |
| **Identidad centralizada** | Samba AD DC (`ad.salud.movil`, realm `AD.SALUD.MOVIL`, DC `dc01`). La usan SMB, Grafana y la consulta de formularios | clinica01 |
| **Backups y restauración** | restic cifrado en un disco USB, en **modelo pull**. kit01 extrae los dumps de cada VM por SSH una vez por semana. Retención de 4 semanales y 3 mensuales | kit01, disco USB |
| **Registros y monitoreo** | Prometheus (node y blackbox exporters para disponibilidad, puertos, CPU, memoria y disco) y Grafana. rsyslog centraliza en kit01 (`/var/log/kitsalud/`, una carpeta por equipo) los eventos de DHCP (Kea), DNS (BIND9), firewall (`fw-drop`) y autenticación (Samba AD y SSH), además del syslog de sw01 | kit01 |

## 2.2 Pruebas de aceptación

| # | Prueba | Cómo se demuestra |
|---|---|---|
| **P1** | Cliente comunitario | `ip a`, `ip -6 route` (`default via fe80::1`), `resolvectl status` / `ipconfig /all` en un cliente de la VLAN 40 |

| # | Prueba | Cómo se demuestra |
|---|---|---|
| **P2** | DNS | `dig A` / `dig AAAA biblioteca.salud.movil`, acceso por navegador sin IP. Extremo a extremo por IPv6 desde la red Interna, con `ping -6 biblioteca.salud.movil` y `curl -6 http://biblioteca.salud.movil` |
| **P3** | Aislamiento | `curl`/`nc` desde la VLAN 40 hacia `pacientes.salud.movil` y hacia el SSH de kit01 → bloqueado (log `fw-drop` en kit01) |
| **P4** | Acceso público local | Desde la VLAN 40, después del portal, `biblioteca`, `videos` y `registro.salud.movil` accesibles |
| **P5** | Portal cautivo | Un cliente nuevo es redirigido a `portal.salud.movil`, también sin Internet |
| **P6** | Aplicación clínica | Un usuario del personal clínico registra y consulta un paciente ficticio en DHIS2 |
| **P7** | Identidad | Un usuario AD inicia sesión en Grafana y en el recurso SMB `archivos` |
| **P8** | Tiempo | `chronyc sources`/`tracking` en clinica01, comunidad01 y una estación de la red Interna apuntando a `ntp.salud.movil` |
| **P9** | Pérdida de Internet | Se desconecta el cable de `wan0` y siguen funcionando DNS, NTP, DHIS2, formularios, biblioteca y videos |
| **P10** | Firewall IPv6 | Desde la VLAN 40, `curl -6` permitido hacia comunidad01 y bloqueado hacia clinica01. Los dos destinos están en la misma red, así que se ve que la regla es por host |
| **P11** | Restauración | Borrar un archivo del recurso `archivos` o una tabla de prueba y restaurarla con `restic restore` desde el disco USB |
| **P12** | Reinicio | Reinicio de un equipo; los servicios `systemd`, las VMs con `autostart` y los contenedores con `restart: unless-stopped` levantan solos |
| **P13** | Diagnóstico | Falla inducida (p. ej. BIND9 detenido o PostgreSQL caído); se localiza con Grafana (sonda blackbox), los logs centralizados y `systemctl`/`journalctl` |

# 3. Supuestos y restricciones

## 3.1 Supuestos

| Supuesto | Impacto si es falso |
|---|---|
| El mini PC Beelink EQi12 (Intel Core i3-1220P de 10 núcleos y 12 hilos, 16 GB DDR4, SSD de 500 GB, **dos NIC de 1 GbE**) es el equipo central del kit | El mismo diseño corre en una laptop de 16 GB con un adaptador USB-Ethernet para la WAN |
| El AP TP-Link TL-WA801ND v3 (firmware 3.16.9) en modo Multi-SSID etiqueta cada SSID con su VLAN (hasta cuatro). Su gestión **recibe** tramas etiquetadas en la VLAN del SSID1 y **responde sin etiqueta** | ether2 de sw01 es híbrido para aceptar esas respuestas (sección 5.2). Si el Multi-SSID con VLAN fallara, la red Interna sería solo cableada |

| Supuesto | Impacto si es falso |
|---|---|
| El uplink entrega IPv4 con salida a Internet (por DHCP o con dirección fija, según el sitio). Si anuncia IPv6, el kit no lo usa, porque IPv6 es interno y nftables no reenvía IPv6 hacia la WAN. En el laboratorio el uplink es la red `192.168.160.0/24` con dirección fija | Si el sitio entrega DHCPv6-PD, se puede agregar GUA más adelante; no cambia las políticas |
| La carga esperada es de 5-10 dispositivos del personal y hasta 50 dispositivos simultáneos de la comunidad | Ampliar el pool de la comunidad a /23 y agregar un AP |
| Todo el software es libre. Windows solo aparece como cliente opcional para unirse al dominio | Ninguno |
| `salud.movil` es un dominio interno, fuera de los TLD públicos, así que no hay certificados públicos | Hace falta una CA interna |
| Todos los datos de pacientes que se usen son ficticios | Ninguno |
| El material propio de la comunidad se graba con ella y con su consentimiento; el kit solo lo almacena y lo publica | Jellyfin sigue publicando el contenido médico general |

### 3.2 Restricciones

| Restricción | Consecuencia en el diseño |
|---|---|
| **No hay UPS física** | Alimentación segura implementada de forma lógica con NUT (`dummy-ups`) y apagado ordenado (sección 13) |
| Un solo mini PC de 16 GB | Dos VMs, router en el host, perfiles por misión (sección 10) |
| Sin Internet en sitio | Todo servicio esencial es local; las actualizaciones se aplican en base |
| El equipo de red es prestado | Las configuraciones se versionan para poder reinstalarlas en cualquier momento |

## 4. Decisiones de diseño

| Decisión | Alternativas descartadas | Justificación |
|---|---|---|
| Kit = **un mini PC (`kit01`)** + MikroTik CCR2004 como switch (`sw01`) + AP multi-SSID (`ap01`) + UPS lógica. Si falta RAM, se agrega una laptop (sección 10.4) | Varios nodos KVM; un servidor físico por rol | Es lo mínimo que cumple el enunciado. Cada VM conserva su IP y su MAC, así que moverla no cambia el direccionamiento, el DNS ni el firewall |

| Decisión | Alternativas descartadas | Justificación |
|---|---|---|
| El **host Ubuntu es el router/firewall**. Nftables, Kea, radvd, BIND9, Chrony y portal cautivo como servicios `systemd` | OPNsense como VM; pfSense; MikroTik | El núcleo de red no depende de ninguna VM. No hay que aprender FreeBSD para el router y se ahorran 3 GB de RAM. Las reglas son archivos de texto versionados, y una sola tabla `inet` aplica a IPv4 e IPv6 |
| El uplink del sitio queda **fuera del kit**. El **CCR2004 trabaja solo en capa 2** (`sw01`), con un bridge con VLAN filtering, una sola interfaz de gestión en la VLAN 10 y el reenvío IP desactivado | CCR2004 como router/firewall del kit | Un solo punto de enrutamiento y de política. Si el CCR2004 también enrutara, habría reglas en dos equipos y el tráfico entre VLAN podría saltarse nftables. Como router, además, separaría las reglas IPv4 de las IPv6 y su portal (Hotspot) solo cubre IPv4. Desconectar el uplink sirve como prueba P9 |
| **Ubuntu Server 24.04 LTS** en el host y en las VMs; KVM/libvirt; aplicaciones en Docker Compose **dentro de las VMs** | Proxmox VE; Docker en el host | Un solo SO y un solo Ansible. Docker en el host reescribiría las reglas del firewall del router; dentro de las VMs no toca el firewall de kit01 |
| Identidad con **Samba AD DC en clinica01**, que también sirve los recursos SMB `archivos` y `contenido` | VM propia de identidad; FreeIPA; OpenLDAP | Cubre el almacenamiento compartido y la identidad con un solo servicio y sin otra VM (+1,5 GB). Es compatible con AD y LDAP para Grafana y la consulta de formularios |
| Pacientes en **DHIS2 con cuentas locales** | OpenMRS; DHIS2 con login LDAP | La sugiere el enunciado y tiene tracker de pacientes e imagen Docker oficial. Si AD falla, el registro de pacientes sigue funcionando |
| Formularios con una **app propia mínima y asistida**. Pocos campos y lenguaje simple; la llena el paciente desde su propio celular, o el personal le ayuda si lo necesita. Los datos se guardan en PostgreSQL de clinica01 con un usuario de solo `INSERT`; la consulta vive en clinica01 con login AD y un usuario de la base que solo lee y marca a quién se atendió | LimeSurvey; formulario de DHIS2 | Un formulario corto se llena rápido desde el celular en la sala de espera. El paciente nunca deja datos en comunidad01, que es la VM expuesta a invitados |
| Contenido comunitario con **Kiwix** (biblioteca de salud y contenido infantil) y **Jellyfin** (audio y video de contenido médico general y para la comunidad) | Kolibri; solo Kiwix | Jellyfin publica cualquier carpeta de audio o video sin preparación previa. Kolibri requiere Kolibri Studio (con Internet) para el contenido propio |
| Observabilidad con **Prometheus, Grafana y rsyslog en kit01** | Zabbix; Loki; monitoreo dentro de una VM | Permite diagnosticar aunque una VM se caiga. Sin Internet no hay a dónde enviar alertas, así que se revisan en Grafana |

| Decisión | Alternativas descartadas | Justificación |
|---|---|---|
| Backups con **restic en modelo pull**. kit01 extrae por SSH los dumps de cada VM y los guarda cifrados en un disco USB | rest-server append-only; push por SFTP | Las VMs no tienen credenciales del repositorio de backups. Es otro medio físico y se desmonta al terminar. Se respaldan datos y configuraciones, no imágenes de VM |
| **Copia en línea** con `restic copy` hacia `rest-server` en modo solo agregar, en un servidor de la organización de salud. Diseñada y sin implementar | Almacenamiento S3 de un proveedor externo; segundo disco como única copia externa | Saca una copia del lugar donde está el kit sin abrir puertos, porque kit01 ya sale por TCP 443. Con solo agregar, un kit comprometido no puede borrar lo que ya subió, y los datos salen cifrados desde kit01 |
| IPv4 `10.20.<id>.0/24`, gateway `.1` | `192.168.<id>.0/24` | Evita solaparse con las redes del uplink (`192.168.x`, como `192.168.160.0/24` en el laboratorio) y se resume en una sola regla `10.20.0.0/16` |
| IPv6 **ULA `fd5a:fc7e:d716::/48`** con un /64 por segmento; sin GUA | Solo GUA; `2001:db8::/32` | Direcciones estables sin Internet. `2001:db8::/32` es solo para documentación (RFC 3849) |
| **Dos NIC sin renombrar.** `wan0` (`enp170s0`) va al uplink y `lan0` (`enp171s0`) es el trunk 802.1Q hacia el switch. `wan0` y `lan0` son nombres de rol en este documento, y la configuración usa los nombres del kernel, que se mantienen entre reinicios | WAN como VLAN 900 en el trunk; renombrar las NIC por MAC | El Beelink EQi12 tiene dos NIC de 1 GbE, así que la WAN queda separada físicamente y desaparece una VLAN. Renombrarlas exigiría un reinicio que, si fallara, cortaría la WAN y el acceso remoto por NetBird sin aportar nada al kit |
| **Dos VLAN físicas** (10 Interna y 40 Comunidad) + **red de servidores virtual** (`br-srv`, un bridge sin puerto físico dentro de kit01) | Cuatro VLAN; tres VLAN con una de gestión aparte | Menos configuración en switch y AP que con cuatro VLAN. El enunciado admite interfaces virtuales y firewalls como mecanismo de separación. Todo el tráfico entre redes pasa por nftables, incluso el que va entre las dos VMs (sección 6.3) |
| **TLS mixto** con una CA propia del kit. La raíz vive solo en kit01 y cada VM tiene un intermedio limitado a sus nombres (*name constraints*), con el que Caddy firma y renueva sus certificados (`tls internal`). El intermedio de clinica01 también firma el certificado LDAPS de `dc01.ad.salud.movil` para Samba AD, que Grafana y la consulta de formularios verifican con la raíz. HTTPS para `registro`, `pacientes`, `consultas`, `archivos` web y `monitoreo`; HTTP para `biblioteca` y `videos`. La raíz se publica en `http://ca.salud.movil` | HTTPS en todo; HTTP en todo lo comunitario; una CA de Caddy independiente en cada VM | El contenido público no lleva datos personales y así se evita la advertencia del navegador. Los formularios sí cifran, y la advertencia se acepta una vez o se instala la raíz, cuyas instrucciones enlaza el portal. Con una sola raíz el personal instala un solo certificado, y si se compromete comunidad01 su intermedio no sirve para falsificar los nombres de clinica01 |
| Credenciales fuera de Git, con `.env.example` y `ansible-vault` | Contraseñas en el README; sops | Cumple "no contraseñas en texto plano" con una sola herramienta |

| Decisión | Alternativas descartadas | Justificación |
|---|---|---|
| **Gestión restringida con un usuario de administración compartido.** kit01, las VMs y sw01 se administran con un solo usuario del grupo (en sw01, el usuario `admin`), por SSH con contraseña y sin acceso directo de root. SSH y GUIs de kit01 solo desde las IPs de administración reservadas en la red Interna y desde NetBird (`wt0`); a las VMs solo se llega a través de kit01 (`ssh -J`). No hay SSID de gestión | Cuentas individuales con llave SSH; VLAN de gestión propia; SSID de gestión | El grupo es pequeño y administra el kit en conjunto, y un solo usuario simplifica el acceso y la automatización. La protección está en el origen (solo IPs de administración y NetBird). Por alcance académico, las contraseñas se mantienen simples y se registran en `ansible-vault`, nunca en los repositorios. Se pierde la trazabilidad por persona (sección 14). Ningún servicio de gestión escucha en la red de la comunidad; el riesgo de compartir VLAN con el personal clínico también está en la sección 14. Las cuentas de las aplicaciones (DHIS2, Samba AD) sí son individuales |
| **Dos VMs separadas por público**. `clinica01` (datos sensibles) y `comunidad01` (lo que ve la comunidad). Se encienden según la misión | Cinco VMs con una IP por rol; todo en el host | Un compromiso de la biblioteca pública no alcanza los datos de pacientes. Cada VM se apaga o se mueve a una laptop sin tocar la otra |
| IPv6 en clientes con **SLAAC + DHCPv6 stateless** (M=0, O=1), **sin RDNSS**. `fe80::1` en cada interfaz interna de kit01 | SLAAC + RDNSS; DHCPv6 stateful | Una sola pareja de mecanismos en todo el kit. DHCPv6 queda demostrado y la ruta por defecto IPv6 es predecible |
| **NetBird solo en kit01** para administrar en remoto | Subnet router de NetBird; WireGuard directo; sin VPN | Funciona detrás del NAT de la universidad sin abrir puertos. Queda fuera de la operación, porque sin Internet no está disponible y el kit no lo necesita |
| Portal cautivo **propio** en kit01. nginx sirve la página de aceptación y un script agrega la MAC del cliente a un set de nftables (8 h) | Portal de OPNsense; portal del AP | Al estar en la tabla `inet`, el mismo set autoriza IPv4 e IPv6. La página del portal sirve además de inicio con íconos hacia biblioteca, videos y registro |
| **Administración remota segura.** kit01 se configura en remoto por NetBird sin tocar la WAN ni NetBird. Los cambios de red se aplican con `netplan generate` y `networkctl reload`, que reconfigura las interfaces sin bajar los enlaces y conserva las direcciones y rutas que no cambian, siempre con una restauración programada. nftables permite siempre NetBird (`wt0` y su tráfico de salida) y SSH, se valida con `nft -c` y se aplica con restauración programada. En sw01 cada cambio remoto se hace en Safe Mode de RouterOS | Configurar solo en sitio; `netplan apply` | Un error en un cambio remoto se deshace solo y no deja al equipo sin acceso |
| **VM de prueba `prueba01`** conectada a `br-com` (VLAN 40) para validar en remoto DHCPv4, IPv6, DNS, portal y firewall desde la red de Comunidad | Probar solo con celulares en el sitio | Casi todas las pruebas de la Comunidad se repiten sin viajar |

# 5. Arquitectura física

## 5.1 Inventario del kit

| Equipo | Rol | Notas |
|---|---|---|
| **`kit01` - mini PC Beelink EQi12** | Router/firewall, servicios de red, hipervisor de clinica01 y comunidad01 | Intel Core i3-1220P (10 núcleos, 12 hilos, hasta 4,4 GHz), 16 GB DDR4, SSD de 500 GB, dos NIC de 1 GbE (`wan0` es `enp170s0` y `lan0` es `enp171s0`, identificadas por MAC porque los puertos no tienen rótulo), fuente interna de 85 W |
| **Disco USB de 500 GB** | Repositorio de backups (restic) | Conectado a kit01; se monta solo durante el backup |
| **MikroTik CCR2004-16G-2S+PC (`sw01`)** | Conmutación L2 con VLAN 802.1Q | RouterOS 7, CPU ARM de 4 núcleos, 4 GB de RAM, 16 puertos de 1 GbE y 2 SFP+ de 10 Gb/s, refrigeración pasiva, alimentación 36-57 V DC. |
| **TP-Link TL-WA801ND v3 (`ap01`)** | Wi-Fi con un SSID por VLAN | Firmware 3.16.9 Build 150723. 802.11n en 2,4 GHz (300 Mb/s nominales), un puerto Ethernet de 10/100, hasta 4 SSID con VLAN, PoE pasivo con inyector incluido (9 V, 0,6 A) |
| **UPS (lógica)** | Alimentación segura y apagado controlado | Sin equipo físico; NUT `dummy-ups` en kit01 (sección 13) |
| **Laptop (opcional)** | Aloja comunidad01 si falta RAM | Solo en el perfil con un equipo más (sección 10.4) |
| ***Fuera del kit:* uplink del sitio** | Salida a Internet (NAT) | En el laboratorio, la red `192.168.160.0/24` conectada a `wan0` |

## 5.2 Conexiones y puertos de sw01

| Puerto | Nombre en RouterOS | Conectado a | Modo | VLAN |
|---|---|---|---|---|
| **ether1** | `ether1-kit01` | kit01 `lan0` (`enp171s0`, MAC `78:55:36:09:07:0a`) | Trunk | 10 y 40 etiquetadas (20 solo en el perfil con un equipo más); solo admite tramas etiquetadas |
| **ether2** | `ether2-ap01` | ap01 | Híbrido | Hacia el AP salen 10 (SSID Clínica y gestión) y 40 (SSID Comunidad) etiquetadas. Desde el AP entra lo etiquetado y, sin etiqueta, lo que va a PVID 10 (respuestas de la gestión del AP) |
| **ether3** | `ether3-laptop` | Laptop (opcional) | Trunk | 10 y 20 etiquetadas; deshabilitado si no se usa |

| Puerto | Nombre en RouterOS | Conectado a | Modo | VLAN |
|---|---|---|---|---|
| **ether4-ether8** | `ether4-interna` … `ether8-interna` | Estaciones clínicas y de administración | Acceso | 10 (PVID 10, solo tramas sin etiqueta) |
| **ether9-ether16, sfp-sfpplus1-2** | - | Sin uso | Deshabilitados y fuera del bridge | - |

El uplink no pasa por sw01 y va directo a `wan0` (`enp170s0`, MAC `78:55:36:09:07:0b`). Los dos puertos Ethernet del mini PC no tienen rótulo, así que cada uno se identifica por el nombre de su interfaz y su MAC.

**Cómo sw01 hace de switch.** En RouterOS, cada puerto es por defecto una interfaz de router independiente. Para que conmute, los puertos en uso se agregan a un **bridge** (un switch dentro del equipo), llamado `bridge-kit`, con `vlan-filtering=yes`. La tabla de VLAN del bridge define qué VLAN lleva cada puerto, con etiqueta o sin ella, y descarta lo que no corresponda (filtrado de ingreso). Los puertos que no se usan quedan deshabilitados y fuera del bridge.

- **Gestión.** La única dirección de sw01 está en la interfaz `vlan10-Interna`, creada sobre el bridge (`10.20.10.2` y `fd5a:fc7e:d716:10::2`, ruta por defecto hacia `10.20.10.1`).
- **Sin enrutamiento.** El reenvío IPv4 e IPv6 está desactivado. Aunque el equipo es un router, no puede enrutar entre VLAN; todo el tráfico entre redes pasa por kit01.
- **CPU en la VLAN 40.** El bridge (`bridge-kit`) es miembro etiquetado de la VLAN 40, sin dirección. ether2 pasa por la CPU (`hw=no`), y sin eso las tramas que llegan de ether1 por el chip no salen hacia el AP y la Comunidad no recibe respuestas.
- **Un solo chip.** Todos los puertos del kit están en el primer chip de switch (`switch1`, ether1-ether8). El segundo chip (`switch2`, ether9-ether16) no conmuta con el primero por hardware y los SFP+ van directo a la CPU; mezclar chips haría pasar ese tráfico siempre por la CPU. Aun así, la CPU alcanza para el tráfico del kit, muy por debajo de 1 Gb/s.
- **Acceso inicial.** Por la consola serial RJ45 (115200 8N1) o por Winbox con la MAC, antes de que exista la VLAN de gestión.

**Protección de sw01**

- Los servicios de gestión de RouterOS (SSH y Winbox) solo aceptan a kit01 y a las IPs de administración; telnet, FTP, web y API se desactivan.
- **DHCP snooping** en el bridge, con ether1 como único puerto de confianza, así que ningún otro puerto puede responder como servidor DHCPv4.
- **RA desde los puertos de acceso.** RouterOS no tiene RA Guard y el filtro del bridge no distingue el tipo de ICMPv6, así que una regla descarta todo el IPv6 hacia `ff02::1` (donde van los RA) que entre

por ether2 y ether4-ether8. El chip de sw01 no acepta reglas de switch y, con conmutación por hardware, el filtro del bridge no ve el multicast, por eso esos puertos pasan por la CPU (`hw=no`); ether1 sigue por hardware. Un RA unicast no se filtra (sección 14).

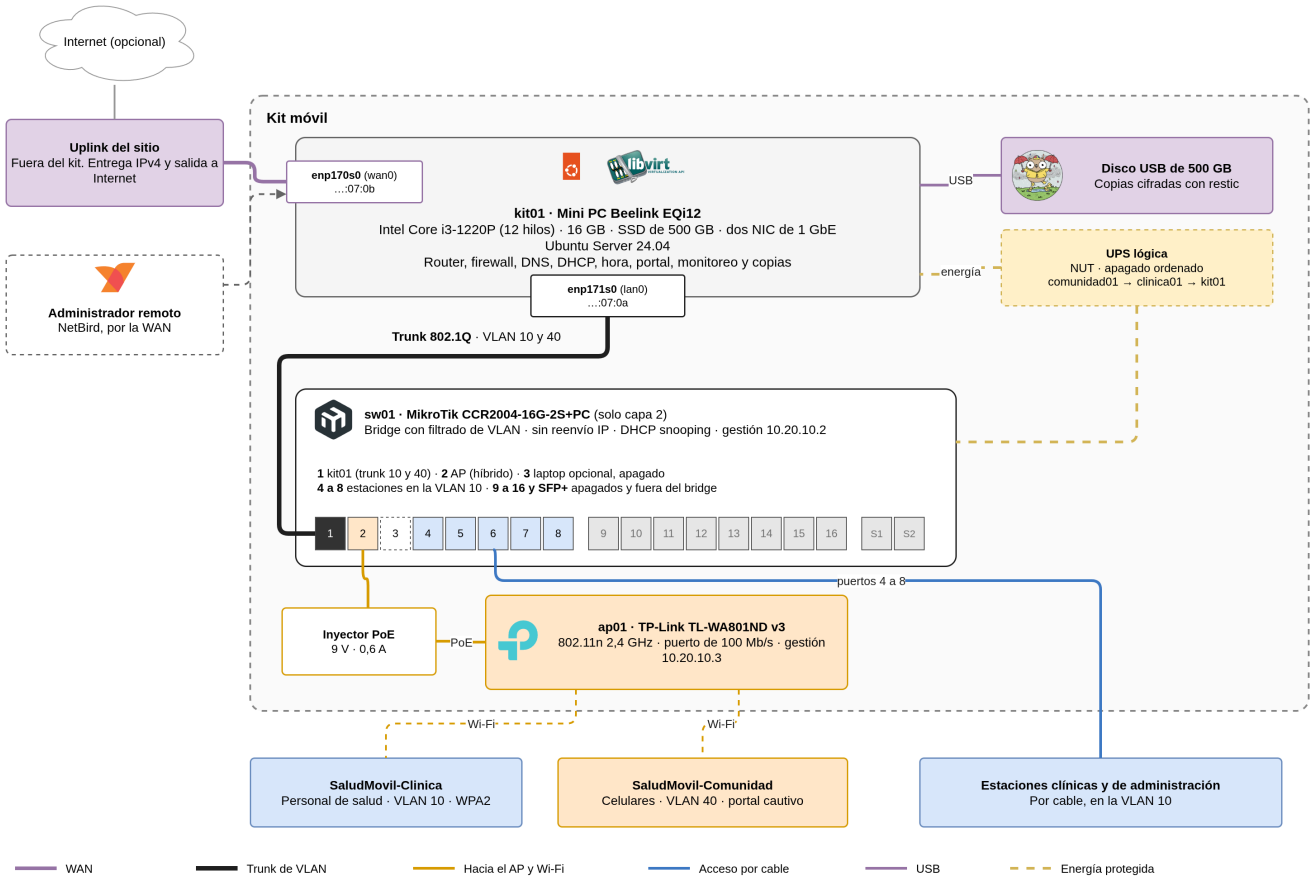
### 5.3 SSID

El AP (firmware 3.16.9) trabaja en modo Multi-SSID con VLAN. El tráfico de cada SSID sale **etiquetado** con su VLAN, y la gestión del propio AP se alcanza desde la VLAN del **SSID1**. La gestión recibe las tramas etiquetadas pero **responde sin etiqueta**; por eso ether2 de sw01 es híbrido, entrega las VLAN 10 y 40 etiquetadas y acepta también tramas sin etiqueta, que mete en la VLAN 10 (PVID 10). Los clientes del SSID Comunidad no pueden aprovecharlo, porque el AP etiqueta su tráfico con la VLAN 40. `SaludMovil-Clinica` es el SSID1 (VLAN 10).

| Posición | SSID | VLAN ID | Seguridad |
|---|---|---|---|
| **SSID1** | `SaludMovil-Clinica` | 10 (Interna; también la gestión del AP) | WPA2-PSK con AES (el AP no soporta WPA3) |
| **SSID2** | `SaludMovil-Comunidad` | 40 (Comunidad) | Abierta + portal cautivo |

- **Dirección del AP.** Fija, `10.20.10.3/24` con gateway `10.20.10.1`. "Allow remote access" queda desactivado, así que la gestión solo responde desde la VLAN 10, que es donde están las IPs de administración. Por NetBird se llega a través de kit01.
- **Servidor DHCP del AP desactivado.** De fábrica viene activo y repartiría direcciones en la VLAN 10; se apaga antes de conectarlo a sw01.
- **Seguridad del AP.** Aislamiento de clientes (*AP Isolation*), que es global y aplica a los dos SSID; WPS y SNMP desactivados.
- **Gestión.** Página web por HTTP, solo en IPv4. Según su documentación, los clientes inalámbricos también pueden llegar a ella. Por alcance académico conserva su contraseña de fábrica (sección 14).
- **Registro.** El AP no envía syslog; su registro se consulta en su propia página.

## 5.4 Diagrama físico



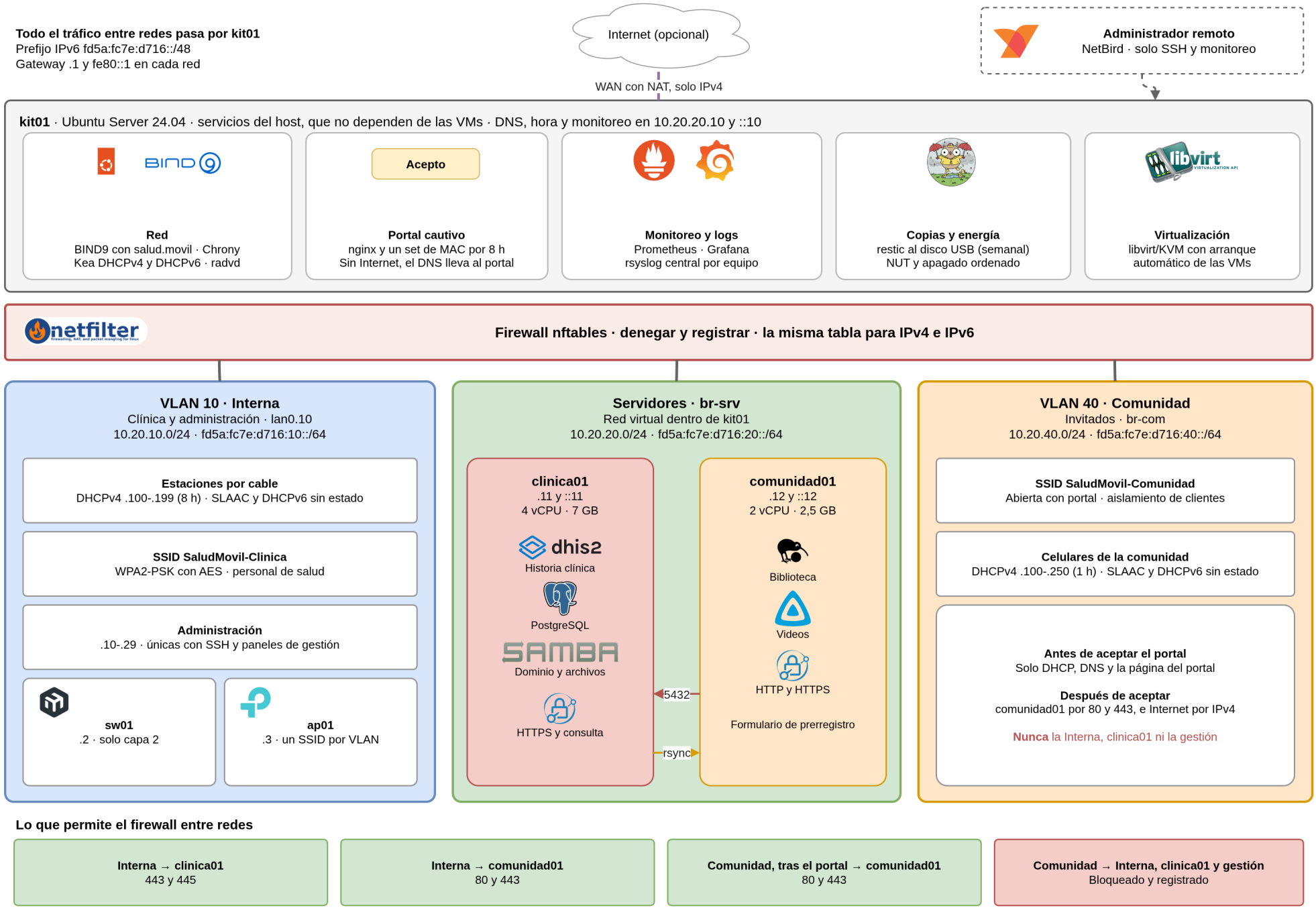

## 5.5 Equipos recomendados

El kit usa los equipos disponibles para el proyecto. Para un despliegue real se recomiendan estos cambios. kit01 se conserva, y los valores salen de las fichas de cada fabricante.

| Componente | En el kit | Recomendado | Por qué |
|---|---|---|---|
| **UPS** | Lógica, con NUT | APC Back-UPS BX1000M (1000 VA, 600 W, 120 V, USB HID) | Protege el kit ante un corte y avisa cuándo apagar. Se conecta por USB y solo cambia el driver de NUT, lo que se comprueba al instalarla |
| **Energía en campo** | Ninguna | Estación LiFePO4 de 1 kWh, como EcoFlow DELTA 2 (1024 Wh, unos 12 kg) | Mantiene el kit una jornada completa en un sitio sin red eléctrica |
| **Punto de acceso** | TL-WA801ND, 2,4 GHz y 100 Mb/s | TP-Link EAP650 (Wi-Fi 6 de doble banda, puerto de 1 GbE, PoE 802.3at, VLAN por SSID y VLAN de gestión) | Quita el cuello de botella del Wi-Fi y permite separar la gestión |

| Componente | En el kit | Recomendado | Por qué |
|---|---|---|---|
| **Switch** | CCR2004, un router de 16 puertos usado en capa 2 | Switch gestionable de 8 puertos con PoE+, como TP-Link TL-SG2210P | Más liviano, y alimenta el AP sin inyector. Antes de comprarlo se comprueba en su ficha el soporte de DHCP snooping y de filtrado de RA |
| **Disco de copias** | Disco de 2,5" a 7200 rpm | Dos SSD USB de 1 TB | Un SSD resiste mejor los golpes del transporte. Uno viaja con el kit y el otro se queda en la base |
| **Memoria de kit01** | 16 GB en dos módulos de 8 GB | 32 GB en dos módulos | Da margen para el crecimiento de clinica01. El equipo reporta hasta 64 GB |
| **Transporte** | Sin definir | Maleta rígida con espuma | Protege los equipos en el viaje. El kit se opera fuera de la maleta para que se ventile |

## 6. Arquitectura lógica


**Todo el tráfico entre redes pasa por kit01**
Prefijo IPv6 fd5a:fc7e:d716::/48
Gateway .1 y fe80::1 en cada red
Internet (opcional)
WAN con NAT, solo IPv4
**Administrador remoto**
NetBird · solo SSH y monitoreo
**kit01** · Ubuntu Server 24.04 · servicios del host, que no dependen de las VMs · DNS, hora y monitoreo en 10.20.20.10 y ::10
**Red**
BIND9 con salud.movil · Chrony
Kea DHCPv4 y DHCPv6 · radvd
Acepto
**Portal cautivo**
nginx y un set de MAC por 8 h
Sin Internet, el DNS lleva al portal
**Monitoreo y logs**
Prometheus · Grafana
rsyslog central por equipo
**Copias y energía**
restic al disco USB (semanal)
NUT y apagado ordenado
**Virtualización**
libvirt/KVM con arranque
automático de las VMs
**Firewall nftables · denegar y registrar · la misma tabla para IPv4 e IPv6**
**VLAN 10 · Interna**
Clínica y administración · lan0.10
10.20.10.0/24 · fd5a:fc7e:d716:10::/64
**Estaciones por cable**
DHCPv4 .100-.199 (8 h) · SLAAC y DHCPv6 sin estado
**SSID SaludMovil-Clinica**
WPA2-PSK con AES · personal de salud
**Administración**
.10-.29 · únicas con SSH y paneles de gestión
**sw01**
.2 · solo capa 2
**ap01**
.3 · un SSID por VLAN
**Servidores · br-srv**
Red virtual dentro de kit01
10.20.20.0/24 · fd5a:fc7e:d716:20::/64
**clinica01**
.11 y ::11
4 vCPU · 7 GB
Historia clínica
PostgreSQL
Dominio y archivos
HTTPS y consulta
5432
rsync
**comunidad01**
.12 y ::12
2 vCPU · 2,5 GB
Biblioteca
Videos
HTTP y HTTPS
Formulario de prerregistro
**VLAN 40 · Comunidad**
Invitados · br-com
10.20.40.0/24 · fd5a:fc7e:d716:40::/64
**SSID SaludMovil-Comunidad**
Abierta con portal · aislamiento de clientes
**Celulares de la comunidad**
DHCPv4 .100-.250 (1 h) · SLAAC y DHCPv6 sin estado
**Antes de aceptar el portal**
Solo DHCP, DNS y la página del portal
**Después de aceptar**
comunidad01 por 80 y 443, e Internet por IPv4
**Nunca** la Interna, clinica01 ni la gestión
**Lo que permite el firewall entre redes**
**Interna → clinica01**
443 y 445
**Interna → comunidad01**
80 y 443
**Comunidad, tras el portal → comunidad01**
80 y 443
**Comunidad → Interna, clinica01 y gestión**
Bloqueado y registrado

### 6.1 Las cuatro redes del enunciado

| Red del enunciado | Cómo se implementa | Interfaz en kit01 |
|---|---|---|
| **Clínica/administrativa** | VLAN 10 **Interna**. Estaciones y SSID del personal | `lan0.10` |
| **Servidores** | **`br-srv`**. Bridge virtual dentro de kit01, sin puerto físico, al que se conectan las VMs | `br-srv` |
| **Comunidad/invitados** | VLAN 40 **Comunidad**. SSID abierto con portal cautivo | `br-com` (con `lan0.40` como puerto) |
| **Administración** | **Plano de gestión**. IPs de administración en la Interna (`10.20.10.10-29`, por reserva DHCP o fijas en el equipo) y el túnel NetBird, que se trata como una extensión de esta red (mismas reglas y mismos usuarios). Son los únicos orígenes con SSH y GUIs; a las VMs solo se llega a través de kit01 | `lan0.10` (filtrada por IP) y `wt0` |

kit01 es el `.1` (y `fe80::1`) de las tres redes internas, así que **todo el tráfico entre ellas pasa por nftables**. La WAN (`wan0`) solo hace NAT de IPv4 hacia el uplink.

### 6.2 Distribución dentro de kit01

- **Núcleo de red (host, `systemd`).** nftables, Kea DHCPv4/DHCPv6, radvd, BIND9, Chrony y el portal cautivo (nginx). Si una VM falla, la red sigue funcionando y los clientes reciben dirección y nombre y ven el portal.
- **Soporte (host).** Prometheus, Grafana, rsyslog central, restic (backups pull), NUT y NetBird.
- **VM `clinica01`.** DHIS2 + PostgreSQL y la consulta de formularios (Docker); Samba AD DC con los recursos `archivos` y `contenido` (nativo).
- **VM `comunidad01`.** Caddy, Kiwix, Jellyfin y la app de formularios (Docker).
- **VM `prueba01`.** Cliente de prueba conectado a `br-com`, solo para validar en remoto la red de Comunidad.
- **Interfaces.** `wan0` es `enp170s0` y `lan0` es `enp171s0`. La VLAN 10 es `lan0.10` y la VLAN 40 es el bridge `br-com`, con `lan0.40` como puerto, para poder conectar ahí la VM de prueba. El único puerto de `br-srv` es la interfaz virtual `srv-dummy0`, que le da portadora para que sus direcciones (entre ellas `10.20.20.10` y `fd5a:fc7e:d716:20::10`) estén disponibles antes de que arranquen las VMs.

Los servicios del host que usan los clientes (DNS, NTP y monitoreo) responden en una IP propia dentro de la red de servidores (`10.20.20.10`). Así, los clientes ven un "servidor de infraestructura" igual que en cualquier red, y si ese rol se moviera a una VM, la IP se iría con él.

### 6.3 Firewall

Una sola tabla `inet` de nftables filtra IPv4 e IPv6 con las mismas zonas (`wan0`, `lan0.10` (Interna), `br-com` (Comunidad), `br-srv` (Servidores) y `wt0`, de NetBird). La política por defecto es **denegar y registrar** (prefijo `fw-drop`). El NAT de salida va en otra tabla (`ip nat_kit`), con una sola regla de masquerade de `10.20.0.0/16` hacia `wan0`, separada del filtrado como pide el enunciado.

NetBird y libvirt usan iptables-nft, que crea sus propias tablas (`ip filter`, `ip nat` y otras). Por eso la tabla de NAT no se llama `ip nat`, el archivo de reglas nunca usa `flush ruleset` (cada tabla del kit se borra y se vuelve a crear en la misma transacción) y detener `nftables.service` solo quita las tablas del kit. Como un paquete tiene que pasar las cadenas de todas las tablas, `inet filtro` acepta el SSH por `wt0` y el túnel de NetBird (`udp/51820`) por `wan0`.

Como `br-srv` vive en kit01, nftables también filtra **entre las dos VMs** (familia `bridge`). El único tráfico permitido entre ellas es el de formularios (comunidad01 → clinica01:5432) y la publicación de contenido (clinica01 → comunidad01:22). Con un switch físico de por medio, ese tráfico no pasaría por ningún firewall.

Este ejemplo muestra cómo se escribe una política equivalente en IPv4 e IPv6.

```
table inet filtro {
  set portal_ok { type ether_addr; flags timeout; timeout 8h; }
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state established,related accept
    iifname "br-com" ether saddr @portal_ok ip  daddr 10.20.20.12           tcp dport { 80, 443 }
accept
    iifname "br-com" ether saddr @portal_ok ip6 daddr fd5a:fc7e:d716:20::12 tcp dport { 80, 443 }
accept
    log prefix "fw-drop " drop
  }
}
```

### 6.4 Portal cautivo

1. Un cliente nuevo de la VLAN 40 solo alcanza DHCPv4, DHCPv6/NDP, DNS (`10.20.20.10`) y la página `portal.salud.movil` en kit01, que se sirve solo por HTTP en `10.20.40.1` y `fd5a:fc7e:d716:40::1`. Una cadena `nat` de prerouting manda a esas direcciones cualquier petición HTTP del cliente, y nginx la lleva al portal. El HTTPS se rechaza con RST, para que el navegador no se quede esperando.
2. El cliente acepta las condiciones. Un servicio de kit01 busca la MAC del cliente en la tabla de vecinos de `br-com`, a partir de su IPv4 o su IPv6, y la agrega al set `portal_ok` por 8 horas. Aceptar otra vez renueva el plazo.
3. Ya autorizado, el cliente llega a comunidad01 (80/443) por IPv4 e IPv6 y, si hay Internet, sale por IPv4 con NAT. No hay salida a Internet por IPv6 (la red es ULA). Nunca llega a la red Interna, a clinica01 ni a los servicios de gestión.

El set, la redirección y las reglas que lo usan están en la tabla `inet filtro`, porque un set solo se usa dentro de su tabla. Así el portal falla cerrado, porque sin el set nadie de la Comunidad pasa de kit01. El set arranca vacío, así que después de un reinicio cada cliente acepta otra vez. Aplicar el firewall conserva las MAC autorizadas con el tiempo que les queda.

Sin Internet, los sistemas operativos no pueden comprobar la conectividad, porque sus nombres de prueba (`connectivitycheck.gstatic.com`, `captive.apple.com` y otros) no resuelven. Para eso kit01 revisa cada 30 s si los reenviadores de BIND9 responden y, con dos fallos seguidos, activa una zona RPZ que responde a la Comunidad cualquier nombre externo con la IP del portal (TTL de 30 s). La Interna, los servidores y las zonas del kit no cambian. Con esa respuesta, el celular abre el portal solo y cualquier dirección por HTTP termina en él, con las condiciones o con el menú según si ya aceptó. Cuando vuelve Internet, la zona queda vacía. El HTTPS externo no se puede redirigir. La página del portal también sirve de **inicio con íconos** hacia biblioteca, videos y registro.

### 6.5 Dependencias entre servicios (orden de arranque)

1. En kit01, red (netplan), nftables, Kea, radvd, BIND9, Chrony, portal.
2. En clinica01, Samba AD DC (necesita tiempo y DNS) → PostgreSQL → DHIS2 y consulta de formularios.
3. En comunidad01, Caddy, Kiwix, Jellyfin, formularios (la app reintenta hasta que responde PostgreSQL de clinica01).
4. De nuevo en kit01, Prometheus, Grafana (login AD contra clinica01; si no responde, queda la cuenta local), backups.

Las dos VMs arrancan con el `autostart` de libvirt, que no tiene un retardo entre VMs. El orden entre servicios lo resuelven las propias aplicaciones, con `depends_on`/healthchecks en Compose, `Restart=on-failure` en systemd y reintentos (la app de formularios espera a PostgreSQL de clinica01).

## 7. Plan IPv4

### 7.1 Segmentos

| Red | Interfaz kit01 | Subred | Gateway | Asignación | Pool DHCP |
|---|---|---|---|---|---|
| **Interna (VLAN 10)** | `lan0.10` | 10.20.10.0/24 | 10.20.10.1 | Estática (red y administración) + DHCPv4 | 10.20.10.100-199 (lease 8 h) |
| **Servidores** | `br-srv` | 10.20.20.0/24 | 10.20.20.1 | Estática | - |
| **Comunidad (VLAN 40)** | `br-com` | 10.20.40.0/24 | 10.20.40.1 | DHCPv4 | 10.20.40.100-250 (lease 1 h) |

| Red | Interfaz kit01 | Subred | Gateway | Asignación | Pool DHCP |
|---|---|---|---|---|---|
| **WAN** | `wan0` | La del uplink (en el laboratorio, `192.168.160.0/24`) | El del uplink (en el laboratorio, `192.168.160.1`) | DHCP o fija según el sitio (en el laboratorio, fija en `192.168.160.69`) | - |

**Convención de hosts.** `.1` gateway, `.2-.9` equipos de red, `.10-.29` servidores (Servidores) o estaciones de administración (Interna), `.100-.250` clientes.

### 7.2 Direcciones de infraestructura

| Host | Función | IPv4 | IPv6 |
|---|---|---|---|
| **kit01** | Gateway de cada red interna | 10.20.10.1 / 10.20.20.1 / 10.20.40.1 | `fe80::1` y `fd5a:fc7e:d716:{10,20,40}::1` |
| **kit01 (servicios)** | DNS, NTP, monitoreo | 10.20.20.10 | fd5a:fc7e:d716:20::10 |
| **sw01** | Switch (gestión) | 10.20.10.2 | fd5a:fc7e:d716:10::2 |
| **ap01** | AP (gestión) | 10.20.10.3 | - (el AP solo se gestiona por IPv4) |
| **clinica01** | DHIS2, Samba AD DC, SMB, consulta de formularios | 10.20.20.11 | fd5a:fc7e:d716:20::11 |
| **comunidad01** | Kiwix, Jellyfin, formularios | 10.20.20.12 | fd5a:fc7e:d716:20::12 |
| **Estaciones de admin** | Reserva DHCP o dirección fija | 10.20.10.10-29 | SLAAC |

## 8. Plan IPv6

### 8.1 Estrategia

- **Prefijo interno.** ULA `fd5a:fc7e:d716::/48`. El Global ID de 40 bits se generó aleatoriamente, como pide la RFC 4193. Es estable y no depende del proveedor.
- **Subredes.** Un /64 por red, y el ID de subred coincide con el tercer octeto IPv4 (`10.20.40.0/24` ↔ `fd5a:fc7e:d716:40::/64`).
- **Servidores.** Direcciones estáticas que espejan el último octeto IPv4 (`10.20.20.11` ↔ `fd5a:fc7e:d716:20::11`).
- **Sin GUA.** IPv6 es solo interno. Con ULA, los clientes prefieren IPv4 para salir a Internet (RFC 6724), así que IPv6 nunca queda como una vía sin portal.

**Vecinos de los clientes Wi-Fi.** Los clientes del Wi-Fi no responden las solicitudes de vecino multicast de kit01, y sí las unicast. Por eso el servicio `kit-vecinos` aprende de cada paquete que entra la dirección y la MAC del cliente, las carga en la tabla de vecinos y el kernel las comprueba por unicast. Solo acepta direcciones del prefijo de cada red.

### 8.2 Asignación por segmento

| Red | Prefijo | Mecanismo | Flags RA | DNS |
|---|---|---|---|---|
| **Interna** | fd5a:fc7e:d716:10::/64 | SLAAC + DHCPv6 stateless (DNS, dominio de búsqueda, NTP) | M=0, O=1 | DHCPv6 (y DHCPv4) |
| **Servidores** | fd5a:fc7e:d716:20::/64 | Estático; sin RA en `br-srv` | - | Estático |
| **Comunidad** | fd5a:fc7e:d716:40::/64 | SLAAC + DHCPv6 stateless | M=0, O=1 | DHCPv6 (y DHCPv4) |

radvd anuncia el prefijo con el flag `A` y el flag `O`, sin RDNSS. Kea DHCPv6 responde los `Information-Request` con el DNS `fd5a:fc7e:d716:20::10`, el dominio `salud.movil` y el NTP. Android no implementa DHCPv6, así que obtiene dirección IPv6 por SLAAC y usa el DNS que recibió por DHCPv4, que también responde AAAA.

### 8.3 Direcciones link-local

| Interfaz | Link-local | Uso |
|---|---|---|
| **kit01** `lan0.10`, `br-com`, `br-srv` | `fe80::1` (fija) | Origen de los RA, gateway IPv6 de clientes (`default via fe80::1`) y servidor DHCPv6 (escucha en `ff02::1:2`, responde desde `fe80::1`) |
| **kit01** `wan0` | Automática (`fe80::/64`) | Sin uso en el kit. Se ignoran los RA que lleguen por la WAN |
| **clinica01, comunidad01** | Automática | Ruta por defecto estática `via fe80::1` en `br-srv` |
| **Clientes y sw01** | Automática | NDP y solicitudes DHCPv6 |

La misma `fe80::1` en varias interfaces es válida, porque una link-local solo tiene sentido dentro de su enlace. Las link-local no se publican en DNS. El firewall las tiene en cuenta (sección 8.4).

### 8.4 Seguridad IPv6

- Las reglas son **equivalentes en v4 y v6**. Cada política tiene su par en la misma tabla `inet` (sección 6.3).
- Se permite el ICMPv6 imprescindible (NDP, RS/RA, Packet Too Big, Time Exceeded, Parameter Problem; RFC 4890) desde `fe80::/10` y desde la ULA. Solo kit01 emite RA (desde `fe80::1`); el AP

aísla a los clientes inalámbricos entre sí y sw01 descarta los RA multicast que entran por los puertos de acceso (sección 5.2).

- DHCPv6 entra por `udp/547` desde `fe80::/10` hacia kit01, en las redes Interna y Comunidad.
- El eco ICMP/ICMPv6 se permite desde la red Interna hacia kit01 y los servidores, y desde la Comunidad solo hacia su gateway. Así se puede demostrar la conectividad extremo a extremo y diagnosticar sin exponer los servidores a la comunidad.
- El portal cautivo autoriza por MAC, así que aplica igual a IPv4 e IPv6.

## 9. DNS y nombres de servicio

Zona autoritativa `salud.movil` en BIND9 (kit01), que escucha solo en `10.20.20.10` y `fd5a:fc7e:d716:20::10`. Las zonas inversas son `20.10.in-addr.arpa` (la de `10.20.0.0/16`) y `6.1.7.d.e.7.c.f.a.5.d.f.ip6.arpa`, con un PTR por dirección. La subzona `ad.salud.movil` está delegada al DNS interno de Samba AD en clinica01, con `dc01.ad` como glue. Sus consultas van siempre a clinica01 (la zona `salud.movil` no usa los reenviadores) y `salud.movil` queda fuera de la validación DNSSEC, porque `movil` no es un TLD real. La recursión solo se permite a `10.20.0.0/16` y `fd5a:fc7e:d716::/48`. Con Internet se usan los reenviadores `1.1.1.1` y `9.9.9.9` (`forward first`, con DNSSEC validado), que no dependen del sitio; sin Internet, las zonas locales siguen respondiendo porque son autoritativas, y la Comunidad recibe la IP del portal para los nombres externos (sección 6.4).

| Nombre | Destino | A | AAAA |
|---|---|---|---|
| **`pacientes.salud.movil`** | clinica01 (DHIS2) | 10.20.20.11 | fd5a:fc7e:d716:20::11 |
| **`archivos.salud.movil`** | clinica01 (SMB) | 10.20.20.11 | fd5a:fc7e:d716:20::11 |
| **`consultas.salud.movil`** | clinica01 (consulta de formularios) | 10.20.20.11 | fd5a:fc7e:d716:20::11 |
| **`dc01.ad.salud.movil`** | clinica01 (Samba AD DC) | 10.20.20.11 | fd5a:fc7e:d716:20::11 |
| **`biblioteca.salud.movil`** | comunidad01 (Kiwix) | 10.20.20.12 | fd5a:fc7e:d716:20::12 |
| **`videos.salud.movil`** | comunidad01 (Jellyfin) | 10.20.20.12 | fd5a:fc7e:d716:20::12 |
| **`registro.salud.movil`** | comunidad01 (formularios) | 10.20.20.12 | fd5a:fc7e:d716:20::12 |
| **`ca.salud.movil`** | comunidad01 (raíz de la CA del kit, por HTTP) | 10.20.20.12 | fd5a:fc7e:d716:20::12 |
| **`ntp.salud.movil`, `ns1.salud.movil`** | kit01 | 10.20.20.10 | fd5a:fc7e:d716:20::10 |
| **`monitoreo.salud.movil`** | kit01 (Grafana) | 10.20.20.10 | fd5a:fc7e:d716:20::10 |
| **`portal.salud.movil`** | kit01 (portal cautivo) | 10.20.40.1 | fd5a:fc7e:d716:40::1 |

| Nombre | Destino | A | AAAA |
|---|---|---|---|
| **`kit01`, `sw01`, `ap01` `.salud.movil`** | Gestión | 10.20.10.1 / .2 / .3 | fd5a:fc7e:d716:10::1 / ::2 / - |
| **`clinica01`, `comunidad01` `.salud.movil`** | Nombre de host de cada VM | 10.20.20.11 / .12 | fd5a:fc7e:d716:20::11 / ::12 |

# 10. Servicios, recursos y perfiles

## 10.1 Catálogo

| Equipo / VM | IP | SO / runtime | Servicios | vCPU | RAM | Disco |
|---|---|---|---|---|---|---|
| **kit01 (host)** | `.1` en cada red, `.10` | Ubuntu Server 24.04 | nftables, Kea DHCPv4/v6, radvd, BIND9, Chrony, nginx (portal y monitoreo), Prometheus, Grafana, rsyslog, restic, NUT, NetBird, libvirt | - | 3 GB (reservados) | 100 GB |
| **clinica01** | `.11` | Ubuntu 24.04 + Docker | DHIS2 (heap de 2 GB) + PostgreSQL/PostGIS, consulta de formularios, Caddy; Samba AD DC con `archivos` y `contenido` | 4 | 7 GB | 40 GB + 120 GB de datos |
| **comunidad01** | `.12` | Ubuntu 24.04 + Docker | Caddy, Kiwix, Jellyfin (sin transcodificación), formularios | 2 | 2,5 GB | 20 GB + 120 GB de medios |
| **prueba01** | DHCP en la VLAN 40 | Ubuntu 24.04 (imagen cloud) | Cliente de prueba (`dig`, `curl`, `ping`) | 1 | 0,5 GB | 10 GB |
| **Total** | | | | **6 vCPU en VMs** | **12,5 GB** | **≈ 400 GB** |

`prueba01` solo se enciende durante las pruebas y no cuenta en los totales ni en los perfiles por misión.

Criterios de las cifras.

- **clinica01.** La guía de DHIS2 pide al menos 2 GB para una instancia pequeña, repartidos entre la JVM y PostgreSQL. Con 5-10 usuarios y Samba AD (≈ 0,5 GB) se asignan 7 GB. La prueba de carga en clinica01 (DHIS2 2.42 con PostgreSQL 16) lo confirma. En reposo, DHIS2 y PostgreSQL usan 1,65 GB; con 5 usuarios registrando 2000 pacientes a la vez llegan a unos 2 GB, y la VM conserva más de 4 GB disponibles. El primer arranque tarda 57 s y un reinicio con datos 25 s.

- **comunidad01.** Jellyfin consume poco si no transcodifica. Los videos se preparan antes en H.264/AAC a 480p (≈ 1 Mb/s, por la capacidad del AP; sección 13) y los audios en MP3, para que se reproduzcan directo.
- **Disco.** qcow2 con aprovisionamiento delgado e independientes (sin archivo base compartido); solo ocupa lo que se escribe (hoy unos 32 GB entre todas las VMs). Viven en un LV propio de kit01 (`ubuntu-vg/vms`, 320 GiB), y el disco de datos de cada VM se monta en `/srv`. Los ZIM de salud e infantiles ocupan unos pocos GB; el resto del espacio es para audio y video.

Todas las VMs llevan `node_exporter`, rsyslog con reenvío a kit01, `chrony` apuntando a `ntp.salud.movil`, SSH solo desde kit01 con el usuario de administración del grupo, sin login directo de root y `ufw` con entrada denegada por defecto (segunda capa detrás de nftables).

### 10.2 Criticidad de los servicios

| Nivel | Servicios | Dónde | Si falla |
|---|---|---|---|
| **1 - Núcleo de red** | Firewall, DHCP, DNS, NTP, portal | kit01 | No hay red y nadie obtiene dirección ni nombres. Por eso vive en el host y no depende de las VMs |
| **2 - Clínico** | DHIS2, consulta de formularios, Samba AD, SMB | clinica01 | No se registran pacientes. La comunidad sigue con biblioteca y videos |
| **3 - Comunitario** | Biblioteca, videos, registro | comunidad01 | La atención clínica sigue; el prerregistro se hace directamente en DHIS2 |
| **4 - Soporte** | Monitoreo, backups, NetBird | kit01 | No afecta la atención; se pierde visibilidad o el respaldo de la semana |

### 10.3 Perfiles por tipo de misión

El mismo kit sirve para misiones distintas, y según la misión se enciende o no cada VM. El núcleo de red (kit01) siempre está encendido.

| Perfil | clinica01 | comunidad01 | SSID activos | RAM usada | Uso |
|---|---|---|---|---|---|
| **Brigada de salud (referencia)** | Sí | Sí | Clínica y Comunidad | ≈ 12,5 GB | Atención médica con sala de espera, prerregistro, biblioteca y videos |
| **Jornada comunitaria** | No | Sí | Comunidad (y Clínica para el personal) | ≈ 5,5 GB | Talleres educativos y biblioteca para niños, sin atención médica. Menos consumo eléctrico. El registro no está disponible y Grafana usa su cuenta local |
| **Solo clínica** | Sí | No | Clínica | ≈ 10 GB | Atención sin servicios para la comunidad |

Cambiar de perfil es `virsh autostart` / `virsh shutdown` de una VM, más activar o desactivar el SSID. El direccionamiento, el DNS y el firewall no cambian.

### 10.4 Perfil con un equipo más

El perfil de referencia alcanza, así que este queda como evolución. Si DHIS2 necesitara más memoria, **comunidad01 se mueve a una laptop** con Ubuntu y KVM.

- La laptop se conecta a ether3 de sw01 (trunk 10 y 20). En kit01, `br-srv` agrega `lan0.20` como puerto y la VLAN 20 lleva la red de servidores hasta la laptop. Es el único caso en que existe la VLAN 20.
- comunidad01 conserva su IP y su MAC. Se copia el qcow2 y la definición (`virsh dumpxml` / `virsh define`), o se redespliega con Ansible.
- El tráfico entre clinica01 y comunidad01 sigue pasando por el bridge de kit01, así que nftables lo sigue filtrando.
- La laptop no se suspende (tampoco al cerrar la tapa) y tiene batería propia.

### 10.5 Contenido para la comunidad

| Servicio | Contenido | Cómo se actualiza |
|---|---|---|
| **Kiwix (`biblioteca`)** | Biblioteca médica en español (Wikipedia Médica), contenido enciclopédico para niños (Vikidia) y simulaciones educativas (PhET), descargados de la biblioteca de Kiwix | Archivos ZIM en la carpeta `zim` del recurso `contenido` |
| **Jellyfin (`videos`)** | Dos bibliotecas. **Salud.** Contenido médico general en audio y video (prevención, higiene, cuidado materno-infantil, primeros auxilios, cuándo consultar). **Comunidad.** Material educativo y para niños y grabaciones de reuniones comunitarias | Archivos de audio y video en las carpetas `salud` y `comunidad` del recurso `contenido` |

**Flujo de publicación.** El personal sube el material al recurso SMB `contenido` de clinica01 con su usuario de AD, en `zim`, `salud` o `comunidad`. Cada 10 minutos, un timer de clinica01 lo copia con `rsync` sobre SSH a la carpeta de medios de comunidad01 (`/srv/medios`), y lo que se borra del recurso también se borra allá. En clinica01 lo ejecuta un usuario local que solo lee el recurso, y un archivo modificado hace menos de 2 minutos espera a la siguiente pasada, por si todavía se está subiendo. En comunidad01 la llave solo se acepta desde clinica01 y solo ejecuta `rsync` en modo demonio, con módulos de solo escritura. Con ella no se abre una shell, no se lee ningún archivo y no se escribe fuera de la carpeta de medios. El flujo va siempre de clínica a comunidad, y comunidad01 no tiene llaves SSH ni credenciales SMB hacia clinica01. Kiwix se reinicia solo cuando cambian los ZIM, y Jellyfin agrega o quita los archivos con su monitor de cambios. Los ZIM de la biblioteca los descarga Ansible en `contenido/zim`. Los audios y videos de la lista los descarga y prepara Ansible directamente en comunidad01, y la publicación nunca los sobrescribe ni los borra. Lo que suba el personal ya viene en H.264/AAC de 480p o en MP3 (sección 10.1). La comunidad ve el contenido sin contraseña, con el usuario `comunidad` de Jellyfin, que solo reproduce y nunca transcodifica. La administración de Jellyfin solo es accesible desde la red Interna y por NetBird a través de kit01. Jellyfin trata esas redes como locales y la cuenta `admin` no tiene acceso remoto, así que desde la Comunidad no puede entrar.

### 10.6 Respaldo

| Qué | Origen | Método | Frecuencia |
|---|---|---|---|
| **BD de DHIS2 y de formularios** | clinica01 | `pg_dump` ejecutado por SSH desde kit01 (usuario `backup` con comando forzado) | Semanal |
| **Dominio Samba AD** | clinica01 | `samba-tool domain backup offline` | Semanal |
| **Recursos `archivos` y `contenido`** | clinica01 | `rsync` pull hacia kit01 | Semanal |
| **Configuraciones (`/etc`, Compose, `.env`)** | kit01, VMs | `rsync` pull | Semanal y antes de cada cambio |

La copia corre el domingo de madrugada. En cada VM la llave de kit01 solo entra desde kit01 y solo ejecuta un comando forzado, que acepta `pg_dump` de las bases de la lista, el backup del dominio y `rsync` en modo demonio con módulos de solo lectura. kit01 junta todo en un área temporal y ejecuta `restic backup` hacia el disco USB, con un snapshot por tipo (configuraciones, recursos, bases y dominio). El repositorio está cifrado y su clave vive en un archivo que solo lee root, desplegado desde `ansible-vault`. El disco se monta por UUID solo durante la ventana de backup, y si no tiene repositorio la primera copia lo crea. La retención es de 4 semanales y 3 mensuales (`restic forget --prune`). Las imágenes de VM no se respaldan, porque se reconstruyen con Ansible y luego se restauran los datos. Los medios de comunidad01 tampoco, porque son una copia de `contenido` o los vuelve a descargar Ansible.

Además de la copia semanal, se hace una copia manual antes de cada cambio y al cerrar cada misión, así que se pierde como máximo una semana de datos.

**Copia en línea.** Está diseñada y sin implementar. Cuando kit01 detecta Internet y la UPS está en línea, sube los snapshots pendientes con `restic copy` a un `rest-server` de la organización de salud, por HTTPS y en modo solo agregar. Los datos salen cifrados y no hace falta abrir puertos, porque kit01 ya sale por TCP 443. La primera subida son 850 MiB, unos 12 minutos a 10 Mb/s, y cada semana sin cambios en los recursos son unos 5 MiB.

### 10.7 Acceso remoto

NetBird corre solo en kit01 (interfaz `wt0`, red `100.64.0.0/10`, que no se solapa con `10.20.0.0/16`). En nftables, `wt0` es una zona más que solo admite SSH a kit01 y HTTPS a `monitoreo`. A las VMs, al switch y al AP se llega con `ssh -J kit01`. La clave de registro de NetBird nunca se versiona. NetBird queda fuera de la operación en campo, porque sin Internet no está disponible y el kit no lo necesita.

### 10.8 Requerimientos de software

Todo sale de los repositorios del Ubuntu o de imágenes con versión fija, así que el mismo kit se puede reconstruir. Son las versiones instaladas en el kit.

| Dónde | Función | Software y versión |
|---|---|---|
| **Todos** | Sistema operativo | Ubuntu Server 24.04.5 LTS |
| **kit01** | Virtualización | QEMU 8.2.2, libvirt 10.0.0 |
| **kit01** | Red | nftables 1.0.9, Kea 2.4.1, radvd 2.19, BIND9 9.18.39, Chrony 4.5, nginx 1.24.0 |
| **kit01** | Monitoreo y logs | Prometheus 2.45.3, blackbox exporter 0.24.0, node exporter 1.7.0, Grafana 13.2.4, rsyslog 8.2312.0 |
| **kit01** | Copias, energía y acceso | restic 0.16.4, NUT 2.8.1, NetBird 0.80.0 |
| **clinica01** | Contenedores | Docker 29.1.3, Compose 2.40.3 |
| **clinica01** | Aplicaciones | DHIS2 2.42.6.0, PostGIS 16-3.5, Caddy 2.11.7, consulta de prerregistros con Flask 3.1.3 |
| **clinica01** | Identidad y archivos | Samba 4.19.5 |
| **comunidad01** | Contenedores | Docker 29.1.3, Compose 2.40.3 |
| **comunidad01** | Aplicaciones | kiwix-serve 3.8.2, Jellyfin 12.3.20261023, Caddy 2.11.7, formularios con Flask 3.1.3 |
| **Máquina de control** | Automatización | ansible-core 2.20.7, `ansible.posix` 2.2.0, `community.general` 12.6.2, `community.crypto` 3.2.2 |
| **Clientes** | Uso | Navegador web. Para `archivos`, un cliente SMB3 |

## 11. Matriz de flujos

La política por defecto es **denegar y registrar**. Todas las reglas aplican a IPv4 e IPv6, salvo que se indique otra cosa. "Admin" son las IPs `10.20.10.10-29` y `wt0`. Los bloqueos quedan en el log con una etiqueta propia (`fw-drop F-18`, `fw-drop F-20` y `fw-drop F-21`) y lo demás con `fw-drop`.

| Origen | Destino | Puertos | Justificación |
|---|---|---|---|
| **Admin** | kit01 | 22, 443 (monitoreo), ICMP | Administración del kit |
| **kit01** | clinica01, comunidad01 | 22 | Salto SSH de administración y backups pull |
| **Interna, Comunidad, VMs** | kit01 `.10` | 53 tcp/udp, 123/udp | DNS y NTP |

| Origen | Destino | Puertos | Justificación |
|---|---|---|---|
| **Interna, Comunidad** | kit01 | 67/udp; 547/udp desde `fe80::/10`; ICMPv6 NDP/RS | DHCPv4, DHCPv6 stateless y descubrimiento de vecinos |
| **Interna** | clinica01 | 443, 445 | DHIS2, consulta de formularios, recursos SMB |
| **Interna** | clinica01 | 88, 389, 464, 636, 135, 49152-65535 | No habilitado. Se agrega solo si se unen equipos al dominio |
| **Interna** | comunidad01 | 80, 443 | Biblioteca, videos, registro y administración de Jellyfin |
| **Comunidad (sin autorizar)** | kit01 | 80 (portal) | Página del portal cautivo, solo por HTTP. El HTTP hacia otros destinos se redirige aquí y el HTTPS se rechaza con RST |
| **Comunidad (autorizada)** | comunidad01 | 80, 443 | Biblioteca, videos y registro. **Única excepción** de la comunidad hacia la red de servidores |
| **Comunidad (autorizada)** | Internet | any, **solo IPv4** con NAT | Conectividad comunitaria, cuando hay Internet |
| **comunidad01** | clinica01 | 5432 | Formularios. `pg_hba.conf` solo acepta a comunidad01, con un usuario de solo `INSERT`. Filtrado en el bridge (tabla `bridge`, con conntrack) |
| **clinica01** | comunidad01 | 22 | Publicación de contenido (`rsync`, usuario restringido). Filtrado en el bridge (tabla `bridge`, con conntrack) |
| **kit01** | clinica01 | 636 | Login LDAPS de Grafana contra AD |
| **kit01** | clinica01, comunidad01 | 9100; 443 y 445 en clinica01; 80 y 443 en comunidad01 | Scraping de `node_exporter` y sondas blackbox de Prometheus. El 5432 de clinica01 ya lo alcanza la sonda TCP y lo filtra `pg_hba.conf`; el 22 y el 636 se sondean por las reglas de administración y de LDAPS. Filtrado en las VMs (ufw) |
| **clinica01, comunidad01** | kit01 `.10` | 514/tcp | Envío de logs (rsyslog), con cola en disco en cada VM. kit01 reconoce cada equipo por su IP de origen |
| **sw01** | kit01 | 514/udp y 123/udp | Syslog y hora del switch. El AP no envía syslog ni tiene cliente NTP |
| **clinica01, comunidad01** | Internet | 80, 443, **solo IPv4** | Actualizaciones de paquetes e imágenes, solo con Internet y en ventana de mantenimiento |
| **kit01** | Internet | 53 y 80 tcp, 443, udp, ICMP | Forwarders DNS, NTP externo, actualizaciones (apt por el 80 y el 443) y túnel de administración. NetBird usa WireGuard, STUN e ICE por UDP con puertos variables, por eso se deja todo el UDP. Lo demás queda bloqueado y registrado (`fw-drop F-18`) |

| Origen | Destino | Puertos | Justificación |
|---|---|---|---|
| **Todas** | Todas | ICMPv6 NDP/PMTU desde `fe80::/10` y ULA | Funcionamiento de IPv6 (RFC 4890) |
| **Comunidad** | Interna, clinica01, gestión de kit01 (22, `monitoreo`) | any | **Bloqueado y registrado** (`fw-drop F-20`) |
| **Interna (no admin)** | kit01 22, VMs 22 | any | **Bloqueado y registrado** (`fw-drop F-21`). SSH solo desde admin |
| **Interna** | kit01, clinica01, comunidad01 | Eco ICMP/ICMPv6 | Diagnóstico y prueba extremo a extremo IPv4/IPv6 |
| **Comunidad, Servidores** | Su gateway (`10.20.40.1` o `10.20.20.1`, su `::1` y `fe80::1`) | Eco ICMP/ICMPv6 | El cliente o la VM verifica su propia conectividad |
| **Interna** | Internet | any, **solo IPv4** con NAT | Conectividad del personal clínico, cuando hay Internet |
| **kit01** | clinica01 | 53 tcp/udp | BIND9 consulta la subzona delegada `ad.salud.movil` al DNS de Samba AD. Filtrado en clinica01 (ufw) |

## 12. Diagnóstico rápido

Procedimiento que otro administrador puede ejecutar.

1. En Grafana (`monitoreo.salud.movil`), sondas blackbox de DNS, web, puertos y ping de cada servicio en IPv4 e IPv6; CPU, memoria, disco y unidades de systemd de kit01 y las VMs. Las alertas activas también se ven en Prometheus (`/classic/alerts`, por un túnel SSH a `127.0.0.1:9090` en kit01).
2. En kit01, `systemctl --failed`, `virsh list --all`, `journalctl -u kea-dhcp4-server -u named -u radvd`, `nft list ruleset` y los logs centralizados en `/var/log/kitsalud/<equipo>/` (`kit01/firewall.log` con los `fw-drop`, `kit01/dhcp.log`, `kit01/dns.log`, `clinica01/auth.log` con los intentos de SSH y de Samba AD, `sw01/syslog.log`).
3. En la VM afectada (`ssh -J kit01`), `docker compose ps`, `docker compose logs`, `systemctl status samba-ad-dc`.

## 13. Dimensionamiento y energía

- **Cómputo.** 6 vCPU en VMs, más los servicios del host, sobre los 12 hilos del i3-1220P (sobreasignación baja). La carga pico es la de DHIS2 durante su arranque y el registro.
- **Memoria.** 12,5 GB de 16 en el perfil de referencia, con ≈ 3,5 GB de margen.
- **Almacenamiento.** ≈ 400 GB asignados en el SSD de 500 GB, de los que hoy hay ocupados unos 45 GB, porque los discos de las VMs son delgados. Copias en un disco USB de 500 GB; hoy ocupan menos de 3 GB, con las imágenes Docker, y crecen con los datos y con los medios de `contenido`.
- **Clientes.** 5-10 del personal y hasta 50 de la comunidad. El pool de la comunidad tiene 151 direcciones con lease de 1 h; el límite real lo pone el AP.
- **Red.** El trunk entre kit01 y sw01 es de 1 GbE. El cuello de botella es el AP, con Wi-Fi 802.11n en 2,4 GHz (300 Mb/s nominales, del orden de 50-80 Mb/s reales compartidos entre todos los clientes) y un puerto Ethernet de 100 Mb/s. Por eso los videos se preparan a 480p (≈ 1 Mb/s), y así caben unas 20-30 reproducciones simultáneas, y con 50 celulares conectados la experiencia depende sobre todo del espacio y la interferencia.

**Energía.** Las cargas son estimadas. Ningún consumo se ha medido con un vatímetro. sw01 solo reporta su voltaje (49,7 V), y los contadores de energía de la CPU de kit01 no incluyen la memoria, el disco, la red ni las pérdidas de la fuente.

| Equipo | Fuente | Capacidad | Consumo típico | Pico estimado |
|---|---|---|---|---|
| **Mini PC Beelink EQi12** | Interna, 100-240 V | 85 W | ≈ 25 W | ≈ 60 W |
| **MikroTik CCR2004-16G-2S+PC** | Adaptador de 48 V y 0,95 A | 45,6 W | ≈ 18 W | 36 W (ficha del fabricante) |
| **AP TL-WA801ND con inyector PoE** | 9 V y 0,6 A | 5,4 W | ≈ 4 W | 5,4 W |
| **Total (brigada de salud)** | | **136 W** | **≈ 47 W** | **≈ 101 W** |

El disco USB se alimenta de kit01 y su consumo entra en el del mini PC. Las laptops tienen batería propia y no entran en la cuenta.

| Dato | Valor | Cómo se obtiene |
|---|---|---|
| **Capacidad instalada** | 136 W | Suma de las fuentes (V × A) |
| **Carga de diseño** | ≈ 95 W | 136 W menos 30 %, porque los equipos no usan toda su fuente al tiempo. El pico estimado de 101 W confirma el orden de magnitud |
| **Carga típica** | ≈ 47 W | Suma de los consumos típicos |

| Dato | Valor | Cómo se obtiene |
|---|---|---|
| **Jornada comunitaria** | ≈ 42 W | Con clinica01 apagada se toma el mini PC en reposo, unos 20 W |
| **Carga térmica** | ≈ 160 BTU/h típica y ≈ 324 BTU/h de diseño | 1 W son 3,41 BTU/h. Equivale a 0,027 toneladas de enfriamiento, así que basta con ventilación natural |

**Autonomía con UPS.** La referencia es una UPS de 1000 VA y 600 W. Su carga de diseño es el 16 % de lo que soporta, así que se elige por energía y no por potencia. El fabricante indica 42 minutos a 100 W, unos 70 Wh útiles, y ese valor se usa para todo el cálculo.

| Carga | Hasta agotarse | Hasta la señal de batería baja (20 %) |
|---|---|---|
| **Típica, 47 W** | Cerca de 1 h 30 | Cerca de 1 h 10 |
| **Diseño, 95 W** | Unos 45 min | Unos 35 min |

**Reserva para apagar.** La señal de batería baja se configura en el 20 %, unos 14 Wh. El peor caso del apagado son 7 minutos, con 180 s por VM y un minuto para kit01, y a carga de diseño usa unos 11 Wh. En las pruebas, cada VM cerró en menos de 5 s.

**Jornada sin red eléctrica.** La UPS cubre un corte y un apagado ordenado, y no alcanza para operar una jornada.

| Caso | Energía necesaria | Equipo | Alcance estimado |
|---|---|---|---|
| **Jornada de 8 h** | 376 Wh típica y 760 Wh de diseño | Estación LiFePO4 de 1024 Wh, unos 870 Wh útiles | Cerca de 18 h a carga típica y 9 h a carga de diseño |
| **Varias jornadas** | Lo mismo cada día | La estación con un panel plegable de 200 W | Unos 640 Wh al día con 4 horas de sol pico |

**Alimentación segura (implementación lógica).** No se cuenta con UPS física, así que el diseño la trata como un componente lógico.

- **Simulación.** En kit01, NUT usa el driver `dummy-ups` con un archivo de estado, que se cambia con `kit-ups ob` (en batería) y `kit-ups lb` (batería baja). En cada arranque el estado vuelve a `OL`, así kit01 no se vuelve a apagar al encender.
- **Apagado ordenado.** `upsmon` ejecuta el script de apagado en el orden comunidad01 → clinica01 (con espera a que PostgreSQL y Samba cierren) → kit01. Cada VM tiene 180 s para cerrar; si no termina, se fuerza para que kit01 alcance a apagarse con batería.
- **Simulacro.** Con un archivo de simulacro reciente, el script apaga las VMs en orden pero no kit01, y la UPS vuelve a `OL`. Así se prueba en el kit sin que nadie tenga que ir a encenderlo.
- **Paso a hardware real.** Con una UPS USB solo se cambia el driver (p. ej. `usbhid-ups`); el procedimiento no cambia.

### 13.1 Qué cambiaría con más usuarios

- **Separar la VLAN de gestión** de la Interna, que es una subinterfaz más en kit01 y un puerto más en el switch.
- **Pasar Samba AD a una VM propia** (+1,5 GB) y usar Samba miembro para los recursos.
- **clinica01 a 10-12 GB**, con la memoria repartida entre la JVM y PostgreSQL.
- **Un segundo nodo** con DNS secundario y réplica de la BD, para quitar el punto único de falla.
- **Un AP de doble banda (Wi-Fi 5 o 6) con puerto gigabit**, o un segundo AP, y el pool de la comunidad a /23.

### 13.2 Sincronización cuando se recupera Internet

El enunciado no exige implementarla, pero sí documentarla. Queda documentada y no implementada.

| Qué | Hacia dónde | Cómo |
|---|---|---|
| **Datos de pacientes (DHIS2)** | Instancia central de DHIS2 de la organización de salud | Trabajos programados de sincronización de datos y metadatos de DHIS2, ejecutados solo cuando kit01 detecta salida a Internet |
| **Formularios de prerregistro** | DHIS2 local | El personal convierte cada prerregistro en un registro del tracker al atender al paciente; viajan al servidor central con el resto de DHIS2 |
| **Contenido (ZIM y medios)** | Desde la biblioteca de Kiwix y un repositorio central de medios | Descarga en base, nunca en campo; se carga en el recurso `contenido` y se publica con el flujo normal (sección 10.5) |
| **Backups** | `rest-server` de la organización de salud, en modo solo agregar | `restic copy` del repositorio del USB por HTTPS, cuando kit01 detecta Internet y la UPS está en línea (sección 10.6) |

## 14. Riesgos aceptados

| Riesgo | Mitigación |
|---|---|
| kit01 es un punto único de falla, porque es el router y el hipervisor | Copias probadas, apagado ordenado, reconstrucción con Ansible y restauración. Un segundo nodo queda como evolución (sección 13.1) |
| sw01 es un router (CCR2004), y un error de configuración (una dirección en otra VLAN o el reenvío activado) lo haría enrutar entre VLAN y el tráfico se saltaría nftables | Reenvío IPv4 e IPv6 desactivado y una sola dirección (la de gestión), con la configuración exportada (`/export`) y versionada |
| DHIS2 consume mucha RAM | Heap limitado a 2 GB y contenedores con límite de 6 GB. La prueba de carga en clinica01 mostró un pico de unos 2 GB con 5 usuarios a la vez. Las tablas analíticas, con unos 220 pacientes y 320 consultas, se generan en 5 s con un pico de 1,9 GB en DHIS2, dentro de su límite. Si con más datos no alcanza, se usa el perfil con un equipo más (sección 10.4) |

| Riesgo | Mitigación |
|---|---|
| El portal cautivo es código propio y la detección de portal falla sin Internet | Lógica mínima (una página, un script y un set de nftables). Sin Internet, una zona RPZ de BIND9 responde a la Comunidad cualquier nombre externo con la IP del portal, así el celular lo detecta |
| Actualizaciones sin Internet | No se actualiza en campo. Se actualiza en la base, en una ventana de mantenimiento, con una instantánea previa de cada VM. Las imágenes Docker tienen versión fija y se guardan con `docker save` en el disco USB |
| La red Interna mezcla personal clínico y administración, así que el switch y el AP son alcanzables dentro de la VLAN y una IP de administración se puede suplantar | Servicios de gestión de sw01 limitados por dirección (`/ip service`), DHCP snooping, RA multicast descartados en los puertos de acceso (los unicast no se pueden filtrar en sw01), SSH solo desde las IPs de administración y NetBird, y registro de accesos. Para crecer, se separa la VLAN de gestión (sección 13.1) |
| Samba AD DC y Docker en la misma VM; Samba recomienda no usar el DC como servidor de archivos en instalaciones grandes | Para el tamaño del kit es aceptable. Si da problemas, AD pasa a su propia VM (+1,5 GB) |
| Material audiovisual con personas de la comunidad | Solo con consentimiento; sin datos de pacientes en los medios; se publica solo en la red local |
| NetBird depende de un servicio externo y de Internet | Solo se usa para administrar en remoto. Se puede desactivar en campo (`systemctl disable netbird`) sin afectar el kit |
| Por alcance académico, el AP conserva su contraseña de fábrica. Además se gestiona solo por HTTP e IPv4 y, según su manual, cualquier cliente inalámbrico puede llegar a su página de gestión | Servidor DHCP propio, WPS y SNMP desactivados y "Allow remote access" apagado, así que la gestión solo responde desde la VLAN 10. En un despliegue real se cambiaría la contraseña y se guardaría en `ansible-vault`. Evolución hacia un AP con VLAN de gestión propia |
| Los clientes del Wi-Fi no responden las solicitudes de vecino multicast, y en la red Comunidad y la Interna dependen de `kit-vecinos` para recibir respuestas por IPv6 | El servicio arranca solo y se reinicia si falla, y las pruebas de IPv6 se ejecutan desde un cliente Wi-Fi real. Un AP distinto se debe probar antes de quitarlo |
| El AP es de 2,4 GHz, 802.11n y con puerto de 100 Mb/s, así que con muchos celulares y video, el Wi-Fi se satura antes que cualquier otro componente | Videos a 480p, aislamiento de clientes y canal automático. Evolución hacia un AP de doble banda con puerto gigabit (sección 13.1) |
| Un solo usuario de administración compartido (kit01, VMs, sw01 y AP) deja sin trazabilidad por persona, y el enunciado recomienda cuentas individuales | Riesgo aceptado por el tamaño del grupo y el alcance académico. Las contraseñas se registran en `ansible-vault`, nunca en los repositorios. Sin acceso directo de root; SSH solo desde las IPs de administración y NetBird; registro de accesos centralizado con la IP de origen. Evolución hacia cuentas individuales con llave SSH |
| Hay una sola copia de respaldo y viaja con el kit, así que un robo, un incendio o un golpe se llevan los datos y el respaldo a la vez | El repositorio está cifrado y kit01 lo monta solo durante la copia. La copia en línea (sección 10.6) y un segundo disco que se rota y se queda en la base completan la regla 3-2-1. Las dos quedan como evolución |

| Riesgo | Mitigación |
| --- | --- |
| No hay UPS física y los consumos no se han medido, así que la autonomía es una estimación | El apagado ordenado está probado con NUT y el cálculo usa el dato del fabricante, que es el más conservador. Con una UPS real se mide la autonomía antes de salir a campo |